

Building an Implicit-Dictionary LZSS Compressor

A Practical Guide to Sliding-Window Matching, Optimal Parsing, and Where the Real Gains Over DEFLATE/zstd/LZMA Come From

How to use this document

This is written as a build guide, not just a theory reference. Each section ends with a concrete deliverable: a data structure, an algorithm, or a file to write. Follow the sections in order – each one only depends on what came before it. Section 12 at the end is the most important one to internalize *before* you start coding: it tells you honestly where the compression-ratio gains over DEFLATE/zstd/LZMA actually come from, so you don't spend weeks on the part that matters least.

Contents

1	Goal and Non-Goal	1
2	Why No Header Dictionary Is Needed: The Causality Argument	2
3	Bounding Memory and Speed: The Sliding Window	3
3.1	Search buffer vs. look-ahead buffer	3
3.2	A fixed-memory search buffer: the ring buffer	3
3.3	Why a smaller, fixed window is also <i>faster</i> , not just smaller	4
3.4	Total memory as a function of your own parameters	5
3.5	Parameter profiles: make the speed/ratio/memory trade-off explicit	5
4	Match Finding	6
4.1	Data structure: hash chains	6
4.2	How deep to walk the chain	8
4.3	Upgrading the matchfinder later: binary trees and suffix automata	8
5	Candidate Representation: Retiring the Old Graph Structure	8
6	Cost Model	9
6.1	What Elias-gamma actually is, as real bits	9
6.2	What it's actually being used for here (and what it isn't)	10
7	Making Near References Cheaper Than Far Ones: Distance/Length Slots	11
8	Optimal Parsing	12
8.1	The flat DP (exactly optimal, given the cost model above)	12
8.2	Extension: repeat-distance modeling (practical, not exactly optimal)	15

9 Entropy Coding Integration	16
9.1 The cost-model / entropy-coder feedback loop	17
9.2 Context choices for the entropy stage, roughly in order of expected payoff	17
10 Implementation Roadmap	18
11 Validation and Benchmarking Protocol	18
12 Calibration: Where the Real Gains Actually Come From	19

1 Goal and Non-Goal

Goal: an LZSS-style compressor where the dictionary is never stored. The decoder reconstructs it implicitly, byte by byte, from the same data it has already decoded – exactly the same way the encoder does. Nothing about a "dictionary" is ever written to the compressed file; only literal bytes and (length, distance) back-references are.

Non-goal: reproducing DEFLATE. DEFLATE's specific parameters (32KB window, greedy/lazy matching, static per-block Huffman) were 1996 engineering trade-offs for speed and memory, not a template for maximum ratio. This guide deliberately departs from several of them; each departure is called out explicitly.

Running example used throughout this document

Every worked example below uses the same 13-byte string, so the pieces connect into one coherent picture instead of thirteen disconnected toy cases:

$$S = \text{a b c a b c a b c x a b c}, \quad \text{positions } 1, 2, \dots, 13.$$

It repeats "abc" three times, is interrupted once by "x", then repeats "abc" once more. Small enough to trace completely by hand, rich enough to show a real match, a real choice between two parses, and a real place where a repeat distance changes.

All bit costs below use exactly the formulas in `lzss_cost.h`: a literal costs 9 bits (1 flag + 8 data bits); a match of length s and distance δ costs $1 + \text{var}(s - 1) + \text{var}(\delta)$ bits, where $\text{var}(x) = 2\lfloor \log_2 x \rfloor + 1$ is the Elias-gamma-style variable-length code cost. Every number quoted below was computed with this exact formula (verified by script, not by hand), so you can recompute any of them yourself as a sanity check while implementing.

2 Why No Header Dictionary Is Needed: The Causality Argument

Definition 1 (Implicit dictionary). *An implicit dictionary at position l is simply the already-decoded prefix $x_{1..(l-1)}$ of the output itself. There is no separate structure to store or transmit.*

Theorem 1 (Decoder can always reconstruct the dictionary on its own). *If the encoder only ever emits back-references whose source position p satisfies $p < l$ (the current position), the decoder can resolve every reference using only bytes it has already produced, in the same left-to-right order the encoder used. No side information about the dictionary is required.*

Proof. By induction on l . At $l = 1$ nothing has been decoded, so only a literal can be emitted (there is nothing to reference). Suppose the decoder has correctly reconstructed $x_{1..(l-1)}$. If the encoder emits a literal at l , the decoder appends it directly. If the encoder emits a back-reference (s, δ) with source position $p = l - \delta$, then $p < l \leq l$, so x_p (and, byte by byte, $x_p, \dots, x_{p+s-1}$, even if $p + s - 1 \geq l$, since each output byte becomes available to be read the instant it is produced) is already in the decoder's output buffer. The decoder copies it forward exactly as the encoder did. By induction, this holds for all l . $\square$

This is the same argument that dissolved the "per-path hash" confusion from earlier in this project: match validity is a property of the *fixed input string*, not of which parse/path was chosen to reach a given position. Section 4 turns this into an algorithm for finding valid matches quickly.

Worked example: decoding without a dictionary

Suppose the encoder emitted, for our running example S : three literals **a**, **b**, **c**; then one back-reference ($s=6, \delta=3$); then a literal **x**; then a back-reference ($s=3, \delta=4$). The decoder replays this with no dictionary at all, just its own output buffer:

1. Emit literals **a**, **b**, **c** $\rightarrow$ buffer is "**abc**".
2. See (6,3): copy 6 bytes starting 3 bytes back from the current end (position $4 - 3 = 1$). Copy byte-by-byte: $\text{buffer}[4] = \text{buffer}[1] = \mathbf{a}$; $\text{buffer}[5] = \text{buffer}[2] = \mathbf{b}$; $\text{buffer}[6] = \text{buffer}[3] = \mathbf{c}$; $\text{buffer}[7] = \text{buffer}[4]$ (just written!) $= \mathbf{a}$; $\text{buffer}[8] = \text{buffer}[5] = \mathbf{b}$; $\text{buffer}[9] = \text{buffer}[6] = \mathbf{c}$. Buffer is now "**abcabcabc**".
3. Emit literal **x** $\rightarrow$ "**abcabcabcx**".
4. See (3,4): copy 3 bytes starting 4 back from position 11 (source position 7): $\text{buffer}[11..13] = \text{buffer}[7..9] = \mathbf{abc}$. Final buffer: "**abcabcabcxabc**" – exactly S .

Note step 2: the source range (positions 1–6) and the destination range (positions 4–9) *overlap*, yet the byte-by-byte copy still works, because each source byte is always already-written by the time it's read (Theorem 1's proof handles this explicitly). This overlapping-copy trick is exactly how a single short back-reference can expand into a long run.

What this replaces

Your current `compressed_header.c` / `code_map.c` store an explicit dictionary. Under the design in this document, those become unnecessary: the header only needs to store per-block metadata (block length, entropy-coder tables if any), not a symbol table. This is the single biggest structural simplification in this rewrite.

3 Bounding Memory and Speed: The Sliding Window

Section 5 already replaces the old `Graph` structure's memory cost (which scaled with $O(n \cdot k)$) with something much smaller. This section does the same thing for the *window* itself: right now, "the window" is implicitly "everything before position l in the current 64,000-byte block" – a *growing* dictionary bounded only by block size, not a true sliding window. That's fine while blocks stay small, but it doesn't give you an explicit, tunable memory ceiling, and it can't reference across a

block boundary. This section fixes both.

3.1 Search buffer vs. look-ahead buffer

Definition 2 (Search buffer and look-ahead buffer). *The search buffer is the set of already-decoded bytes eligible as copy sources – this is your window, and its size is what you want to bound. The look-ahead buffer is just the next few not-yet-encoded bytes the matchfinder is trying to extend a match into; it only needs to be as large as the longest match you’re willing to consider (Section 5’s note about not hard-capping match length still applies – just cap it at something sane, e.g. a few hundred to a few thousand bytes, not thousands of times the window).*

Only the search buffer needs active memory-bounding; the look-ahead buffer is small and fixed by construction.

3.2 A fixed-memory search buffer: the ring buffer

Definition 3 (Ring-buffer window). *Choose a window size W (a power of two, for cheap modular arithmetic). Store the search buffer as `buffer[W]`; the byte at absolute stream position i lives at `buffer[i % W]`. Writing a new byte at position i simply overwrites whatever was at `buffer[i % W]` – the byte from position $i - W$, which has fallen out of the window and can never be referenced again.*

This gives you $O(W)$ memory for the search buffer, *independent of total file size* – exactly the “fixed memory based on parameters I control” property you’re after.

Enforcing the window bound in the matchfinder. Recall Algorithm 2’s hash-chain walk: `prev[p]` always points to a *strictly earlier* position, so distance $i - p$ only grows as the chain is walked. That gives you a free, correct early-exit:

Stop walking the chain the moment $i - p > W$.

Every remaining entry in the chain is even farther away, hence already outside the window and unusable – this is both a correctness requirement (you must never reference an evicted byte) and a speed win (you stop doing work you’d have to discard anyway). You never need to explicitly delete old chain entries; capping the walk at distance $> W$ makes them naturally unreachable.

Worked example: watching eviction happen

Take a deliberately tiny window, $W=4$, and trace the ring buffer through our running example $S = \text{abcabcabcxabc}$ (slot = $(i-1) \bmod 4$):

i	byte	slot	buffer contents after write (slots 0–3)	evicted
1	a	0	a _ _ _	–
2	b	1	a b _ _	–
3	c	2	a b c _	–
4	a	3	a b c a	–
5	b	0	b b c a	byte from $i=1$ (a)
6	c	1	b c c a	byte from $i=2$ (b)
7	a	2	b c a a	byte from $i=3$ (c)
$\vdots$				
11	a	2	...	

By position 5, the byte written at position 1 is already gone – overwritten, not just “old.” This is exactly what “fixed memory regardless of file length” costs you: any match wanting to reference something more than W bytes back simply cannot exist, because the byte is no longer there to compare against or copy from.

Why this makes W a real, consequential choice, not just a memory knob. Recall the match found at $i=11$ in Section 4: length 3, distance 4 (source position 7). With $W=4$, distance $4 \leq W$, so this match is still exactly on the edge of legality – it survives. With $W=3$ instead, distance $4 > W=3$, and this exact match would be *unavailable*: the matchfinder’s chain walk would stop before ever reaching position 7 (Algorithm 2’s $i - p \leq W$ check), and the parser would be forced to fall back to literals there instead, costing real bits (compare $d[13]=54$ in Section 8’s worked table against what it would become without that match – strictly worse). This is the concrete mechanism behind Section 3.5’s claim that increasing W can only help or tie the ratio: a bigger window never removes a match that a smaller one could find, it can only add matches a smaller window would have missed.

3.3 Why a smaller, fixed window is also *faster*, not just smaller

Your instinct that “an algorithm using more memory tends to be slower” is correct, and it’s worth understanding precisely *why*, since it tells you what to tune. The mechanism is CPU cache locality, not memory quantity per se:

- A window/hash-chain structure that fits inside the CPU’s L2 or L3 cache (typically a few hundred KB to a few tens of MB, depending on hardware) gets match-finding lookups served in a handful of cycles.
- Once the structure is larger than cache, lookups increasingly miss into main memory, each costing on the order of *hundreds* of cycles instead – a slowdown of one to two orders of magnitude per access, not a gentle degradation.
- This is exactly why real compressors expose a window/level parameter instead of always using the largest window possible: past a certain size, you are trading a large, predictable speed cost for a shrinking ratio gain (most of the exploitable repetition in typical data lives within a much smaller distance than the theoretical maximum).

So "fixed, appropriately-sized memory" isn't just easier to reason about – choosing W to comfortably fit your target cache tier is a legitimate, direct speed lever, separate from and in addition to `max_chain`.

3.4 Total memory as a function of your own parameters

With window size W and a hash table indexed by `hash_bits` bits (so $2^{\text{hash_bits}}$ buckets):

Structure	Size	Notes
Ring buffer (search buffer data)	W bytes	Section 3.2
<code>head[]</code> (hash $\rightarrow$ most recent position)	$4 \cdot 2^{\text{hash_bits}}$ bytes	one <code>uint32_t</code> per bucket
<code>prev[]</code> (chain links)	$4 \cdot W$ bytes	one <code>uint32_t</code> per window position
Total (encoder)	$W + 4 \cdot 2^{\text{hash_bits}} + 4W$	$= O(W)$, independent of file size
Total (decoder)	W	only the ring buffer – no hash chain needed at all

Note the last row: the *decoder* only ever needs the ring buffer – decoding is pure sequential copying (Section 2), never search. All the matchfinder memory (`head`, `prev`) is encoder-only. If you later upgrade to a binary-tree matchfinder (Section 4), `prev[]` becomes two child pointers per position instead of one, i.e. $8W$ instead of $4W$ – still $O(W)$, just a larger constant.

Worked example: memory for a 1MB window

$W = 2^{20}$ (1,048,576 bytes), `hash_bits` = 18 (262,144 buckets):

ring buffer = 1MB

`head[]` = $4 \times 262,144 = 1\text{MB}$

`prev[]` = $4 \times 1,048,576 = 4\text{MB}$

total (encoder) $\approx 6\text{MB}$, regardless of whether the input is 1MB or 1GB.

The decoder needs only the 1MB ring buffer.

3.5 Parameter profiles: make the speed/ratio/memory trade-off explicit

Rather than picking W , `hash_bits`, and `max_chain` once and hardcoding them, expose them as named profiles (mirroring how `gzip`'s `-1..-9`, `zstd`'s levels, and `xz`'s presets work) so you can dial ratio vs. speed without touching code:

Profile	W	<code>hash_bits</code>	<code>max_chain</code>	Encoder memory (approx.)
Fast	64 KB	15	32	≈ 448 KB
Balanced	1 MB	18	128	≈ 6 MB
Max ratio	16 MB	20	4096 (or unbounded)	≈ 68 MB

These are starting points to benchmark from (Section 11), not fixed recommendations – the right values depend on your actual data and hardware. The point is that all three levers (W , `hash_bits`, `max_chain`) are things *you* set, not properties of the input, which is exactly the “control the parameters, don’t let memory grow with data” property you asked for.

Relationship to Section 12’s “Fast” and “Max ratio” endpoints

Increasing W can only ever *help or tie* compression ratio (a larger window means strictly more candidate sources), never hurt it – so the ratio side of this trade-off is monotonic. The speed cost is what’s non-monotonic (cache locality, Section 3.3) and chain-walk cost (`max_chain`), which is why these are worth exposing as explicit, benchmarked profiles rather than a single guess.

4 Match Finding

4.1 Data structure: hash chains

Definition 4 (Hash chain matchfinder). *For every position i , hash the next H bytes ($H = 3$ or 4 is standard) into a table `head[hash]` pointing to the most recent position with that hash, with each position also storing `prev[i]` pointing to the next-most-recent earlier position sharing the same hash – a singly linked list per hash bucket, walked nearest-first.*

Algorithm 1 Insert position i into the hash-chain structure

```

1: procedure INSERTPOSITION( $i$ ,  $data$ ,  $head[]$ ,  $prev[]$ )
2:    $h \leftarrow \text{Hash}(data[i..i + H - 1])$ 
3:    $prev[i] \leftarrow head[h]$ 
4:    $head[h] \leftarrow i$ 
5: end procedure
```

Algorithm 2 Find match candidates ending at position l (equivalently, starting at $i = l - s + 1$ for the position under consideration)

```

1: procedure FINDMATCHES( $i$ ,  $data$ ,  $head[]$ ,  $prev[]$ ,  $max\_chain$ ,  $W$ )
2:    $h \leftarrow \text{Hash}(data[i..i + H - 1])$ 
3:    $p \leftarrow head[h]$ 
4:    $best\_len \leftarrow 0$ 
5:    $candidates \leftarrow \emptyset$   $\triangleright$  list of (length, distance), one per achievable length
6:    $tries \leftarrow 0$ 
7:   while  $p \geq 0$  and  $tries < max\_chain$  and  $i - p \leq W$  do  $\triangleright$  window bound, Section 3.2
8:      $len \leftarrow \text{MatchLength}(data, p, i)$   $\triangleright$  extend while bytes match, capped by window
9:     if  $len > best\_len$  then
10:      for  $s \leftarrow best\_len + 1$  to  $len$  do
11:         $candidates[s] \leftarrow (s, i - p)$   $\triangleright$  nearest distance for every new length reached
12:      end for
13:       $best\_len \leftarrow len$ 
14:    end if
15:     $p \leftarrow prev[p]$ 
16:     $tries \leftarrow tries + 1$ 
17:  end while
18:  return  $candidates$ 
19: end procedure

```

Key property this relies on

Because the chain is walked nearest-first, the *first* time a given length is reached during the walk, its distance is automatically the smallest (cheapest) distance achieving that length. You never need to compare distances for the same length – just keep walking until longer matches stop appearing, or the chain/budget runs out.

Worked example: two hash-chain lookups on S

Take $H=1$ (hash on a single byte, for simplicity of the example only – real implementations use $H=3$ or 4 to avoid degenerate chains). As positions 1, 2, 3 (a,b,c) are inserted, $head['a'] = 1$, $head['b'] = 2$, $head['c'] = 3$.

Query at $i = 4$ ('a'): $h = 'a'$, $p = head['a'] = 1$. $\text{MATCHLENGTH}(data, 1, 4)$ extends: $S[4]=a=S[1]$, $S[5]=b=S[2]$, $S[6]=c=S[3]$, $S[7]=a=S[4]$ (self-overlap, fine), $S[8]=b=S[5]$, $S[9]=c=S[6]$, $S[10]=x \neq S[7]=a$ – stop. $len = 6$. $prev[1]$ is empty (no earlier 'a'), so the chain ends after one try. Result: candidates (2, 3), (3, 3), (4, 3), (5, 3), (6, 3) – one entry per length from 2 up to the max found, all at distance 3 (Section 5 shows exactly this list).

Query at $i = 11$ ('a', after $head['a']$ has since been updated to 7, then to... actually 7 is the most recent 'a' before 11, since $i = 4$'s 'a' was itself inserted before $i = 7$'s): $p = head['a'] = 7$ (nearest first, so we try 7 before the older 4 or 1). $\text{MATCHLENGTH}(data, 7, 11)$: $S[11]=a=S[7]$, $S[12]=b=S[8]$, $S[13]=c=S[9]$, end of string – stop. $len = 3$, distance = $11 - 7 = 4$. Because the chain is walked nearest-first, we never even need to check $p = 4$ (distance 7) or $p = 1$ (distance 10): both would give the same length-3 match at a needlessly larger distance, hence strictly more expensive under the cost model in Section 6. Result: candidates (2, 4), (3, 4).

4.2 How deep to walk the chain

`max_chain` trades encode speed for match quality. DEFLATE's higher settings use depths in the hundreds; since your goal is ratio, not encode speed, use a generous depth (thousands, or unbounded for smaller blocks like your current 64,000-byte block size – exhaustive chain walking on 64KB is entirely affordable on modern hardware).

4.3 Upgrading the matchfinder later: binary trees and suffix automata

Hash chains can still miss the best match if the chain is very long and gets truncated by `max_chain`. Two standard upgrades, in order of typical benefit-to-effort:

- **Binary-tree matchfinder** (what LZMA's strongest mode, BT4, uses): replace each hash bucket's linked list with a binary search tree ordered by the matched suffix, giving exact nearest-distance-per-length without a depth cutoff, at $O(\log n)$ insertion/search instead of $O(\text{chain length})$.
- **Suffix automaton / suffix array**: gives you the true longest previous factor at every position in one linear-time pass over the whole block, as discussed earlier in this project. More work to implement correctly, but removes match-finding quality as a variable entirely.

Start with hash chains (Algorithm 2); it is by far the easiest to get correct, and with a generous `max_chain` on a 64KB block it will already be very close to exhaustive.

5 Candidate Representation: Retiring the Old Graph Structure

Your existing `Graph/GraphNode` machinery enumerates every possible chunk length $1..k$ at every position (the "virtual parent link" design), because the old dictionary scheme needed to consider every candidate length to know which ones repeat anywhere in the file. A real matchfinder does not need this: at each position it hands you a short, explicit list of *real* candidates.

Definition 5 (Candidate list). *For each position $l = 1, \dots, n$, define C_l as the list of pairs (s, δ) returned by `FINDMATCHES` – typically a handful of entries, not up to $k = 128$.*

Recommendation: replace `Graph/GraphNode/MAX_GRAPH_NODES` with a flat, per-position array of candidates:

```
typedef struct {
    uint32_t length;
    uint32_t distance;
} MatchCandidate;

typedef struct {
    MatchCandidate candidates[MAX_CANDIDATES_PER_POSITION];
    uint8_t count;
} PositionCandidates;

PositionCandidates candidates[BLOCK_SIZE]; // one entry per position
```

This is smaller, simpler, and removes `SEQ_LENGTH_LIMIT` as an artificial cap on match length – a real match can be as long as the matchfinder actually finds (DEFLATE's 258 and LZMA's 273

caps are themselves just wire-format choices, not fundamental limits; choose a cap based on your entropy coder’s length-encoding range, not before).

Worked example: the candidate lists for S

Continuing the running example, the non-empty candidate lists produced by FINDMATCHES across all of S are just the two lookups from Section 4, plus one more (a match starting at $i = 7$ against the nearer source $p = 4$, found the same way):

i	candidates (<i>length</i> , <i>distance</i>)	source
1–3, 10	none (first occurrence of each byte)	–
4	(2, 3), (3, 3), (4, 3), (5, 3), (6, 3)	$p = 1$
7	(2, 3), (3, 3)	$p = 4$ (capped by x at position 10)
11	(2, 4), (3, 4)	$p = 7$ (nearest ‘a’ before 11)

All other positions (2, 3, 5, 6, 8, 9, 10, 12, 13) have empty candidate lists of their own at that *starting* position (though they may still be *covered* by a match that started earlier, e.g. position 6 is covered by the $i=4$ match if the parser chooses to use it – see Section 8, where this exact table becomes the input to the DP).

6 Cost Model

You already have the right building block: `lzss_cost.h`’s `lzss_match_cost_bits(length, distance)` and `lzss_literal_cost_bits()`. Nothing about this section changes once real distances are available – the cost model was already written distance-aware, precisely in anticipation of this rewrite.

6.1 What Elias-gamma actually is, as real bits

The formula $\text{var_cost}(x) = 2\lfloor \log_2 x \rfloor + 1$ isn’t an abstract estimate pulled out of nowhere – it’s the exact bit-length of a real, simple, self-delimiting binary code for positive integers, called an *Elias-gamma code*. It’s worth seeing what it actually looks like, since “self-delimiting” is doing real work here: the decoder needs to know where one number’s encoding ends and the next begins, without a separate length field, and this is the mechanism that provides that.

Definition 6 (Elias-gamma code). *To encode a positive integer n : write n in binary, note $k = \lfloor \log_2 n \rfloor$ (the number of bits after the leading 1), then emit k zero bits, followed by a single 1 bit, followed by those k remaining bits verbatim. Total length: k zeros + 1 bit + k bits = $2k + 1$ bits – exactly $\text{var_cost}(n)$. To decode: count zero bits until the first 1 (that count is k), then read the next k bits as the remainder r ; the value is $n = 2^k + r$.*

Worked example: the actual bits for values from our running example

n	binary of n	k	Elias-gamma code (actual bits)
1	1	0	1 (1 bit)
3	11	1	011 (3 bits) – the distance used in the running example
4	100	2	00100 (5 bits) – the other running-example distance
500	111110100	8	00000000111110100 (17 bits) – the "far" option

Decoding 011: count zeros before the first 1 $\rightarrow k=1$; the first 1 is the stop marker; read the next $k=1$ bit as the remainder, $1 \rightarrow r=1$; value = $2^1 + 1 = 3$. Correct, and the decoder never needed to be told in advance how many bits were coming.

6.2 What it's actually being used for here (and what it isn't)

This is worth being precise about, since Elias-gamma plays *two conceptually separate* roles that are easy to blur together:

1. **Its real, current job: a cost estimate for the parser.** The DP in Section 8 needs *some* number representing "how expensive is this token" in order to compare parses. Elias-gamma is used here *because its shape is right* – cost grows with $\log(\text{value})$, so small distances and short lengths are correctly treated as cheap and large ones as expensive – not because the DP is required to actually emit Elias-gamma bits into your compressed file. As Section 9 already says, your real entropy coder will very likely beat this estimate (an adaptive model over distance *slots*, Section 7, learns your data's actual distribution instead of assuming this fixed shape) – that's exactly why the iterative refinement loop exists, to keep the DP's decisions aligned with whatever your real coder actually costs.
2. **An optional, legitimate second job: a real "phase 0" bitstream encoder.** Because Elias-gamma is a genuine, complete, working binary code – not just a cost formula – you can, if you want a fully working end-to-end encoder/decoder before building the adaptive range coder in Section 9, literally write these bits to the file (one flag bit per token, then a byte for literals or Elias-gamma-coded length/distance for matches). This genuinely saves space compared to fixed-width fields (a 3-bit distance costs 3 bits, not a fixed 16 or 32), just not as much as an adaptively-modeled coder eventually will.

Why building the "phase 0" encoder first is worth doing, not just a shortcut

If you implement Elias-gamma *as your real bitstream*, something useful falls out for free: the DP's predicted cost $d[n]$ (Section 8) and your actual compressed file size will match *exactly* (up to rounding to whole bytes) – because they're now computing the literal same thing. That equality is a strong, easy correctness check for your matchfinder-plus-DP pipeline (Sections 4–8) in isolation, *before* the extra complexity of an adaptive entropy coder is anywhere in the picture. Once that's verified, swap in the real range coder from Section 9 – at that point $d[n]$ becomes an estimate again rather than an exact prediction, and the refinement loop is what keeps it honest.

Reminder: why this is exactly the right unit now

Under the old dictionary/header scheme, a chunk’s true cost depended on how many distinct dictionary entries the whole file ended up needing – a shared, global, history-dependent cost the flat DP could not see, which is why the DP lost to greedy in your benchmark. Under pure back-references, a match’s cost is a function of its own (*length*, *distance*) alone, nothing shared with any other token. This is exactly the assumption the DP’s optimality proof requires, so the same DP that lost to greedy before will not have that problem here.

7 Making Near References Cheaper Than Far Ones: Distance/Length Slots

DEFLATE and LZMA both avoid spending a full, uniformly-costed field on every distance: instead they bucket distances into a small number of *slots* covering exponentially growing ranges, entropy-code the (small) slot index, and store the position *within* the slot as close to raw bits as possible, since values within a bucket are close to equally likely.

You already have a version of this – here’s the connection

`lzss_cost.h`’s `var_cost(x) = 2⌊log2 x⌋ + 1` is already an Elias-gamma code, and Elias-gamma *is* a slot scheme: it splits x into a slot index $k = \lfloor \log_2 x \rfloor$ (written in unary, costing $k+1$ bits) and k raw extra bits locating x within that slot. So the DP’s cost estimates are already slot-shaped, and nothing here changes how the *parser* reasons about near vs. far. What changes is the *entropy coder*: replacing Elias-gamma’s fixed unary slot-index code with an *adaptively modeled* one, learned from your actual data’s distance distribution, rather than assumed to follow a fixed geometric-ish shape.

Definition 7 (Distance slot). *For a distance $\delta \geq 1$, let $k = \lfloor \log_2 \delta \rfloor$ (the slot index). Slot k covers the range $[2^k, 2^{k+1} - 1]$ and needs exactly k extra bits to specify which value in that range: $extra = \delta - 2^k$. The same construction applies to match lengths (over a much smaller range, since matches are typically far shorter than the window).*

Worked example: slotting our running example's distances

δ	slot k	extra bits (value)	
1	0	0 bits	(smallest possible distance)
2	1	1 bit (value 0)	
3	1	1 bit (value 1)	the distance used in our running example's big match
4	2	2 bits (value 0)	the distance used in our running example's trailing match
500	8	8 bits (value 244)	the "far" option in the greedy-vs-optimal counterexample

For a 64KB window, the largest possible distance is 65,536, whose slot is $k=16$ – so there are only **17 distinct slot values** (0 through 16) no matter how large δ gets within that window. That's the real payoff: instead of an entropy coder having to learn a probability distribution over up to 65,536 raw distance values, it only ever needs to learn one over 17 slot values – small enough to adapt quickly and cheaply, even early in a file, while the (much larger, but nearly-uniform) extra bits are left mostly raw, where an adaptive model would buy little anyway.

Where this belongs in your pipeline: the slot/extra-bit split itself is a representation choice for the *entropy coder* (item 3), not the parser – encode slot index with an adaptive model (a small frequency table over ~ 17 –20 symbols is cheap to maintain and converges fast), and write extra bits close to raw (LZMA adds one adaptively-modeled "aligned" bit per slot on top of raw bits as a small extra refinement; worth trying once the basic scheme works, not before). Once this real coder exists, feed its *actual measured* per-slot costs back into the DP through the iterative refinement loop in Section 9 – exactly the same loop already in this guide, just with a better cost source than the Elias-gamma estimate.

8 Optimal Parsing

8.1 The flat DP (exactly optimal, given the cost model above)

This is precisely Section 7 of `doc/writeup.tex`, restated for candidate lists instead of graph levels:

Algorithm 3 Optimal parse via forward DP

```
1: procedure OPTIMALPARSE(data, candidates[], n)
2:    $d[0] \leftarrow 0$ 
3:   for  $l \leftarrow 1$  to  $n$  do
4:      $d[l] \leftarrow d[l-1] + \text{LITERALCOSTBITS}()$ 
5:      $back[l] \leftarrow \text{LITERAL}$ 
6:     for each  $(s, \delta) \in \text{candidates}[l]$  do
7:        $cand \leftarrow d[l-s] + \text{MATCHCOSTBITS}(s, \delta)$ 
8:       if  $cand < d[l]$  then
9:          $d[l] \leftarrow cand$ 
10:         $back[l] \leftarrow (\text{MATCH}, s, \delta)$ 
11:       end if
12:     end for
13:   end for
14:   return RECONSTRUCTPATH(back, n)
15: end procedure
```

Complexity: $O(n \cdot \bar{c})$ time, $O(n)$ space, where $\bar{c}$ is the average candidate-list length (typically small, bounded by `max_chain` or by how many distinct lengths were actually found – not by k). This is the direct successor of `shortest_path_dp.c`: same forward-sweep structure, simpler inner loop (no `get_parent_level/useless` bookkeeping needed), same optimality proof.

Worked example: the full DP table for S

Feeding the candidate table from Section 5 into OPTIMALPARSE gives (literal = 9 bits always; match costs from Section 6's formula, e.g. $\text{match_cost}(6, 3) = 9$, $\text{match_cost}(3, 4) = 9$, $\text{match_cost}(2, 4) = 7$):

l	$d[l]$ (bits)	chosen edge
0	0	–
1	9	literal a
2	18	literal b
3	27	literal c
4	36	literal a (no match reaches exactly here yet)
5	32	match(2, 3) from $d[3] = 27$
6	34	match(3, 3) from $d[3] = 27$
7	34	match(4, 3) from $d[3] = 27$
8	36	match(5, 3) from $d[3] = 27$
9	36	match(6, 3) from $d[3] = 27$
10	45	literal x
11	54	literal a
12	52	match(2, 4) from $d[10] = 45$
13	54	match(3, 4) from $d[10] = 45$

Final parse: literal(**a**), literal(**b**), literal(**c**), match(6, 3), literal(**x**), match(3, 4) – total $27 + 9 + 9 + 9 = 54$ bits, matching Section 2's decode example exactly.

Why row 9 is the interesting one. There are two genuinely different ways to reach position 9: one big match (6, 3) covering positions 4–9 in a single token (36 bits total), or two smaller matches – (3, 3) from $i=4$ reaching position 6, then (3, 3) again from $i=7$ (source $p=4$) reaching position 9 ($d[6] + \text{match_cost}(3, 3) = 34 + 7 = 41$ bits). The DP correctly rejects the split (41 bits) in favor of the single long match (36 bits): every extra token pays a fresh flag bit and a fresh length/distance code, so fragmenting an otherwise-contiguous match is a real, measurable cost (5 bits here) with no offsetting benefit, since both pieces would have used the same distance anyway. This is exactly the kind of decision a greedy "always take what's in front of you" parser gets right by luck when there's only one source available (as here) – but not always, as the next example shows.

Worked counterexample: where plain greedy actually loses

This one is a constructed synthetic scenario (not from S), built specifically to isolate the failure mode. Suppose at some position, the matchfinder’s chain search finds one long-but-far match: length 7, distance 500. Suppose also that, purely by coincidence, taking just *one literal byte* here and then re-querying the matchfinder one position later would reveal a match covering the remaining 6 bytes at distance 4 (very near) – but a plain greedy encoder, which only ever asks “what’s the best match available *right now*”, cannot see this: it has no mechanism to look one step ahead and re-evaluate.

Option	Tokens	Cost
A (plain greedy’s choice)	one match(7, 500)	$1 + \text{var}(6) + \text{var}(500) = 1 + 5 + 17 = 23$ bits
B (what the DP finds)	literal + match(6, 4)	$9 + (1 + \text{var}(5) + \text{var}(4)) = 9 + 11 = 20$ bits

Option B wins by 3 bits, purely because a distance of 4 is so much cheaper to encode than a distance of 500 that sacrificing one byte of match length more than pays for itself. Greedy takes Option A because it is the longest match *visible from the current position* and greedy never reconsiders; the DP in Algorithm 2–8 naturally finds Option B because $d[]$ is computed for *every* position and compared globally – it doesn’t matter how many steps of “lookahead” the better option required, since the DP has no lookahead limit at all. Even DEFLATE’s “lazy matching” (which checks only one position ahead) would fail on a version of this example requiring two or more steps of foresight; the DP has no such bound.

8.2 Extension: repeat-distance modeling (practical, not exactly optimal)

Real data reuses the *same* back-reference distance repeatedly (column-aligned data, periodic structure). LZMA exploits this with 4 cached “recent distances” (`rep0..rep3`); matching one of them costs far fewer bits than encoding a fresh distance.

Honesty check

Tracking recent distances exactly turns the DP state into (position, exact set of recently used distances) – and distances range up to the window size, so this state space is not small in general. *Exact* joint optimization of parse and repeat-distance cache is not tractable this way, and there is no widely known polynomial algorithm for it in full generality (it is closely related to the “smallest grammar” family of hard problems flagged earlier in this project). Real encoders, including LZMA’s own optimal-parse mode, do not solve this exactly either – they use a bounded-lookahead approximation. What follows is that same standard, practical approximation, not a new exact algorithm.

The practical approximation (what LZMA and zstd’s optimal-parse modes actually do): extend the DP state to $(l, \text{rep0})$, where `rep0` is *only the single most recently used distance along the best path reaching l* (not all four, and not all combinations) – carried forward from `back[1]` the same way `shortest_path_dp.c` already carries forward a single winning predecessor per level. At each position, in addition to the matchfinder’s candidates, check whether `rep0` (inherited from the predecessor state) still matches at this position; if so, offer it as an extra candidate with a cheaper cost (no full distance encoding needed, just a “reuse `rep0`” flag):

Algorithm 4 Optimal parse with a single repeat-distance slot

```
1: procedure OPTIMALPARSEWITHREP0(data, candidates[], n)
2:    $d[0] \leftarrow 0$ ;  $rep[0] \leftarrow 0$ 
3:   for  $l \leftarrow 1$  to  $n$  do
4:      $d[l] \leftarrow d[l-1] + \text{LITERALCOSTBITS}()$ ;  $back[l] \leftarrow \text{LITERAL}$ ;  $rep[l] \leftarrow rep[l-1]$ 
5:     for each  $(s, \delta) \in \text{candidates}[l]$  do
6:        $cand \leftarrow d[l-s] + \text{MATCHCOSTBITS}(s, \delta)$ 
7:       if  $cand < d[l]$  then
8:          $d[l] \leftarrow cand$ ;  $back[l] \leftarrow (\text{MATCH}, s, \delta)$ ;  $rep[l] \leftarrow \delta$ 
9:       end if
10:    end for
11:     $r \leftarrow rep[l-1]$ 
12:    if  $r > 0$  and  $\text{MATCHLENGTHAT}(data, l-r, l) > 0$  then
13:       $s' \leftarrow \text{MATCHLENGTHAT}(data, l-r, l)$ 
14:       $cand \leftarrow d[l-s'] + \text{REPMATCHCOSTBITS}(s')$  ▷ cheaper: no distance field
15:      if  $cand < d[l]$  then
16:         $d[l] \leftarrow cand$ ;  $back[l] \leftarrow (\text{REPMATCH}, s')$ ;  $rep[l] \leftarrow r$ 
17:      end if
18:    end if
19:  end for
20:  return RECONSTRUCTPATH(back, n)
21: end procedure
```

This stays $O(n \cdot \bar{c})$ – one extra $O(1)$ -ish check per position – and typically captures most of the practical benefit of repeat-distance caching. Extending to 2–4 tracked slots (closer to LZMA’s `rep0..rep3`) is a straightforward generalization of the same idea if benchmarking shows it is worth the added complexity; start with one slot and measure before adding more.

Worked example: what rep-distance caching would (and wouldn’t) save here

In our running example’s optimal parse, the first match uses distance 3, and the second uses distance 4 – *different* distances, so a `rep0` cache would not help between them (this is a realistic, honest outcome, not a cherry-picked success case). But suppose a variant of *S* had no "x" interruption, so the trailing "abc" also ended up matching at distance 3 (the same as the first match). Encoding that trailing match fresh costs $\text{match_cost}(3, 3) = 7$ bits as usual; but if the encoder instead recognizes "this is the same distance as last time" and emits a cheap reuse flag instead of a full distance code, the cost drops to just $1 + \text{var}(2) = 1 + 3 = 4$ bits (flag + length code only, no distance field at all) – a 3-bit saving on that one token alone. Across a file with many aligned repeats at the same distance (e.g. column-structured or periodic data), these per-token savings compound into one of the largest practical wins available, which is why Section 12 flags this as often the best ratio-per-effort item on the whole roadmap.

9 Entropy Coding Integration

Your plan (item 3: range coding / context mixing) is already the right choice – strictly more expressive than DEFLATE’s Huffman stage, which is limited to integer-bit code lengths per symbol.

9.1 The cost-model / entropy-coder feedback loop

The bit costs in Section 6 are *estimates* (an Elias-gamma-style approximation), not what your adaptive entropy coder will actually spend, since its costs depend on model state that evolves with the actual token sequence chosen. The standard, well-tested resolution (used by LZMA/zstd) is iterative:

1. Parse once using the static cost estimates from Section 6.
2. Feed the resulting token sequence through the real entropy coder; measure the actual bits it assigned to each literal/length/distance symbol.
3. Re-derive per-symbol cost estimates from those measured statistics.
4. Re-run OPTIMALPARSE (Section 8) with the updated costs.
5. Repeat 2–4 rounds, or until total size stops improving.

This is not a proof of global optimality (exact joint optimization of parse and adaptive model state is intractable in general, for the same reason as Section 8.2) – but it is an empirically excellent fixed point, and it is exactly what today’s best general-purpose compressors ship.

9.2 Context choices for the entropy stage, roughly in order of expected payoff

1. Separate probability models for literals vs. match-length vs. distance symbols (do not share one model across all three).
2. Condition literal probabilities on the preceding 1–2 bytes (order-1/order-2 context), not just an unconditional byte frequency table.
3. Condition on whether the previous token was a literal or a match (DEFLATE and LZMA both find this genuinely useful).
4. If you pursue full context mixing: combine several such predictors with a logistic mixer (PAQ/cmix-style) rather than picking one context order – this is where most of the ratio gap to state-of-the-art compressors actually lives, more so than in parsing sophistication.

10 Implementation Roadmap

#	Deliverable	Notes
1	Hash-chain matchfinder over a ring-buffer window	Algorithm 2 plus Section 3.2. New file, e.g. <code>matchfinder.c</code> . Pick a starting profile from Section 3.5.
2	Candidate-list data structure	Section 5. Replaces <code>Graph/GraphNode</code> for this purpose.
3	Flat optimal-parse DP	Adapt <code>shortest_path_dp.c</code> to iterate over candidate lists; reuse <code>lzss_cost.h</code> as-is (already written for this).
4	"Phase 0" Elias-gamma bitwriter (optional but recommended)	Section 6.2. Write the actual Elias-gamma bits computed by the cost model to the file. Gives you a complete, working encoder/decoder fast, and $d[n]$ will exactly match your real file size – a strong correctness check before adding entropy-coder complexity.
5	Round-trip test harness	Verify byte-exact decompression on your existing <code>tests/</code> corpus before optimizing further.
6	Benchmark vs. <code>gzip/zstd/xz</code>	Establish your real baseline gap <i>before</i> adding complexity – see Section 11.
7	Repeat-distance (rep0) DP extension	Section 8.2. Only after 1–6 are working and measured.
8	Distance/length slot encoding	Section 7. Part of the entropy coder, item 9 below, not the parser.
9	Entropy coder + refinement loop	Section 9. Likely your single biggest remaining ratio lever – see Section 12.

11 Validation and Benchmarking Protocol

1. **Correctness first, always:** every change gets a full round-trip (compress → decompress → byte-diff) over your entire `tests/` corpus, including `tests/cantrbry`, before you look at compression ratio at all.
2. **Compare against real baselines on the same files:** `gzip -9`, `zstd --ultra -22`, and `xz -9e` on every test file, not just your previous version – your own history-vs-history comparisons (DP vs. greedy) are useful for internal regressions, but the external baselines are what tells you whether you are actually competitive.
3. **Ablate one change at a time:** matchfinder quality, DP vs. greedy, rep-distance, entropy stage – toggle each independently so you know which one is actually earning its keep, the same way this document's earlier greedy-vs-DP benchmark isolated exactly one variable.
4. **Track encode time too:** an exhaustive `max_chain` and a full context-mixing entropy stage can get slow; note the trade-off even if ratio is your primary goal, so you know what you're spending to get it.

12 Calibration: Where the Real Gains Actually Come From

Read this section before writing code, not after.

1. **Matchfinder quality** (Section 4): a shallow hash chain vs. an exhaustive one, or vs. a binary tree, changes what matches you can even see. Real, but bounded, gains.
2. **Optimal vs. greedy parsing** (Section 8): you already measured this empirically on your own data once the cost model is fair (see the earlier greedy-vs-DP benchmark in this project) – meaningful, but this alone will bring you roughly to where LZMA/zstd’s own optimal-parse modes already are, not past them, since they do this too.
3. **Repeat-distance modeling** (Section 8.2): often the single best ratio-per-effort win on structured/periodic data, and a real point of leverage over compressors that skip it.
4. **Entropy coding sophistication** (Section 9): once parsing is near-optimal, this is realistically where most of the remaining gap to state-of-the-art ratios lives – how well you predict the next symbol’s probability, not how you choose chunk boundaries. The very best general-purpose compressors (cmix, paq-family) get their ratio almost entirely from rich, mixed context models here, not from unusual match-finding.

Honest expectation setting

Beating gzip/DEFLATE outright is a realistic, achievable target with just Sections 4–8 (better matchfinder + optimal parse already exceeds DEFLATE’s greedy/lazy design). Matching or narrowly beating zstd at high levels and competing with LZMA/xz is achievable but will require Sections 8.2–9 done well, since those tools already run near-optimal parsers themselves – your edge over them has to come from matching or exceeding their entropy-coding sophistication, not from parsing alone. Decisively beating LZMA/zstd/xz *in general*, across all data types, is a very high bar few tools clear; beating them on a *specific* data type you care about (by exploiting structure they don’t model) is a much more tractable and still valuable goal. Aim there first.